

ARCHITECTURE & WORKFLOW GUIDE

Arora Classes — How It All Works

Architecture and workflows for the website of Arora Classes, a Class 9–12 coaching centre in Kitchlu Nagar, Ludhiana. Each section shows what happens, where in the system it happens, and the exact request and response that make it work.

NEXT.JS + SUPABASE

FREE TIER — ₹0/MONTH

2 TABLES · 1 ADMIN

5 WORKFLOWS

Contents

1. The problem, and what the site does about it
2. The big picture — three tiers
3. Two ways data travels
4. Workflow — a parent finds information
5. Workflow — a parent submits an enquiry
6. Workflow — the owner logs in
7. Workflow — the owner posts an update with a PDF
8. Workflow — the owner works the enquiry inbox
9. Frontend layer — routes and rendering
10. Backend layer — pipeline and endpoints
11. Database layer — tables, bucket, DDL
12. Security — three layers of defence
13. Validation — one schema, both sides
14. Deployment — where each part runs
15. Key decisions and roadmap

SECTION 1

The problem, and what the site does about it

Arora Classes receives 10–15 enquiry calls a day. Most arrive during class hours and go unanswered; the ones that connect ask the same four things. Parents who can't get through call the next centre.

What happens today

- Phone rings during a lecture — nobody picks up
- Same four questions repeated all day: timings, fees, free seats, faculty
- Notices are told to students verbally and forgotten at home
- No record of who enquired, so nobody is followed up

What the site changes

- Timings, fees and faculty are published, so most calls never happen
- An enquiry form captures the rest, even at 11 p.m.
- The owner posts notices himself, with a PDF if needed
- Every enquiry is stored with a status, so none is lost

How success is measured. Before launch, note roughly how many calls arrive daily and how many are missed during class hours. After two weeks live, count the enquiries received through the form. "Six enquiries in two weeks that would previously have been missed calls" is a real result — and measuring it is what separates a project from a demo.

The big picture — three tiers

Every action in the system passes through these tiers, in order. The client tier is untrusted — everything a visitor's phone downloads is public. The application tier holds all logic and every secret. The data tier stores text and files, and sends email.

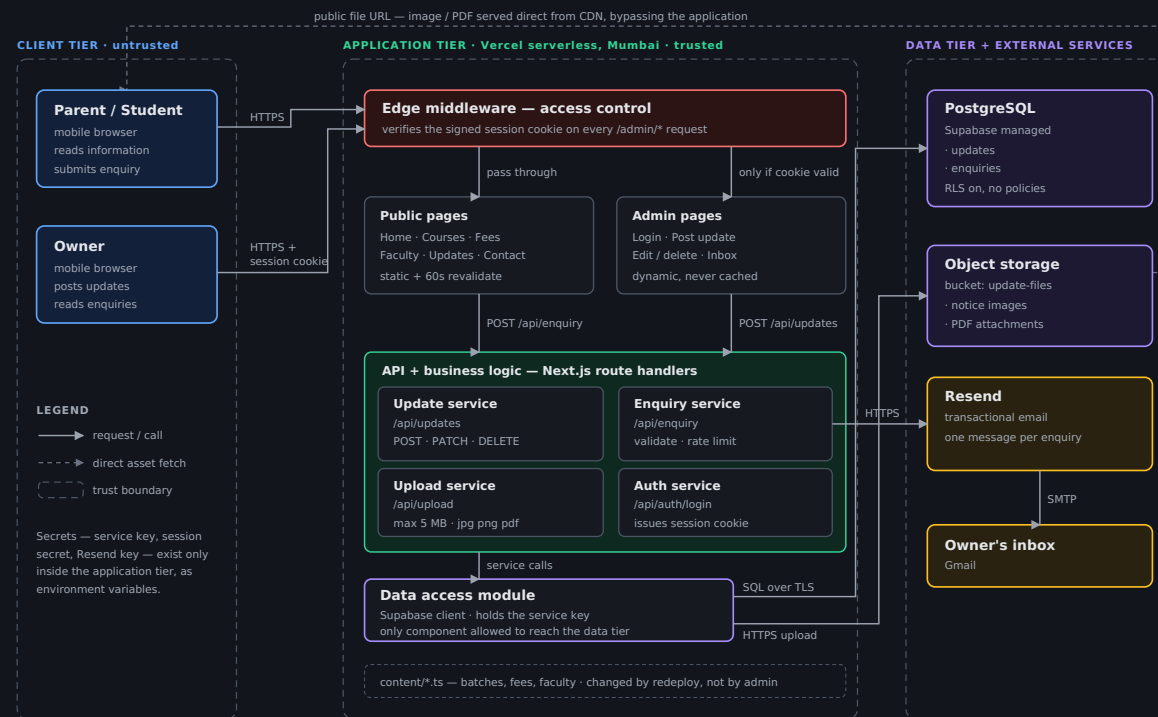


Figure 1 — Component and deployment view with trust boundaries.

How to read the diagrams

- **Dashed rectangles** are trust boundaries.
- **Solid arrows** are requests, labelled with protocol and payload.
- **Dashed arrows** are direct asset fetches that bypass the application.
- **Monospaced labels** such as `POST /api/enquiry` are the exact endpoint.

Two ways data travels

This is the most important concept in the document. Every request in the system follows one of two paths, and which one it takes decides whether a server is involved at all.

Path A — Static & CDN (most traffic)

The page was built in advance and is served from the CDN edge. No server runs, no database is touched, nothing can fail at request time.

- Home, Courses, Fees, Faculty
- Notice images and PDFs, fetched straight from storage
- The Updates page between revalidations

Path B — API routes (every write)

The browser calls a serverless function, which validates the input and uses the service key to reach the database. Every write in the system takes this path.

- Submitting an enquiry
- Logging in
- Creating, editing or unpublishing an update
- Uploading an image or PDF
- Changing an enquiry's status

Why two paths? Path A costs nothing and cannot break, so everything that can be static is static. But the service key that reaches the database must never be exposed to a browser — so anything that writes has to go through Path B, where the key stays server-side.

A parent finds information

The most common journey, and the one that removes the most phone calls. No server is involved at any point.

1 Parent opens the site

She searches for the centre's name or taps a link from a WhatsApp message. The page is already built and cached at the CDN edge, so it arrives in a few hundred milliseconds even on 4G.

PATH A · STATIC

GET / · no server, no database

```
// app/courses/page.tsx – built once at deploy time
import { batches } from '@content/batches';

export default function Courses() {
  return <BatchTable rows={batches} />;
}

// content/batches.ts – edited in code, not by the owner
export const batches = [
  { cls: '11', subject: 'Physics', days: 'Mon-Sat',
    time: '6:30–8:00 pm', seats: 'Filling' },
  { cls: '12', subject: 'Chemistry', days: 'Mon-Sat',
    time: '8:00–9:30 pm', seats: 'Open' },
];
```

2 She checks the Updates page

This page is regenerated at most once a minute. Between regenerations every visitor gets the same cached copy, so a hundred parents opening it cost one database read, not a hundred.

PATH A · REVALIDATED

GET /updates → GET /api/updates

```
// app/updates/page.tsx
export const revalidate = 60; // seconds

export default async function Updates() {
  const rows = await getPublishedUpdates();
  return rows.map(u => <UpdateCard key={u.id} {...u} />);
}

// Response – 200
{ "ok": true, "data": [
  { "id": "6f1c...", "title": "Diwali break: 20–24 Oct",
    "body": "Classes resume Monday 25th.",
    "file_url": null, "created_at": "2026-10-14T05:20:11Z" }
]}
```

3 She opens the attached PDF

The file comes straight from object storage over the CDN. It never passes through the application, which is why serverless functions never have to stream binaries.

PATH A · DIRECT

GET /storage/v1/object/public/update-files/...

```
// The database row holds only the URL
{
  "title": "Class 12 pre-board timetable",
  "file_url": "https://xxxx.supabase.co/storage/v1/
    object/public/update-files/
    updates/9a4f2c17-....pdf"
}

// The browser fetches that URL directly.
// No API route, no function invocation, no cost.
```

A parent submits an enquiry

The core write path of the whole system, and the first thing to build. It crosses four components: public page → enquiry service → data access module → PostgreSQL, with a branch to the email service.

1 She fills the form and taps Submit

Five fields, no more — every extra field loses submissions. The button disables immediately, which is what stops double submissions.

PATH B · PUBLIC

POST /api/enquiry

```
// Request
{
  "name": "Rajwinder Kaur",
  "phone": "9876543210",
  "student_class": "11",
  "message": "Is the evening physics batch open?",
  "website": "" // honeypot – bots fill this
}
```

2 The server checks it before anything else

Rate limit first, then schema, then the honeypot. Browser-side validation is a convenience only — it is repeated here because anyone can call the endpoint directly.

SERVER

rate limit → Zod → honeypot

```
const EnquirySchema = z.object({
  name: z.string().min(2).max(60),
  phone: z.string().regex(/^[6-9]\d{9}$/),
  student_class: z.enum(['9', '10', '11', '12', 'other']),
  message: z.string().max(500).optional(),
  website: z.string().max(0),
});

// 400 – validation failed
{ "ok": false, "error": {
  "code": "INVALID_PHONE",
  "message": "Enter a 10-digit mobile number" } }
```

3 The row is written

Only the data access module touches the database, and it is the only place the service key exists. Row Level Security blocks every other route in.

DATA ACCESS

INSERT INTO enquiries

```
// lib/db.ts – service key, server-side only
export async function createEnquiry(input: Enquiry) {
  const { data, error } = await supabase
    .from('enquiries')
    .insert({ ...input, status: 'new' })
    .select('id, created_at')
    .single();

  if (error) throw new DbError(error.message);
  return data;
}
```

4 The owner is emailed — and failure is contained

Email is the least reliable step in the system. It runs *after* the row is saved, inside its own try/catch, so a delivery failure never fails the request.

SIDE EFFECT

Resend → owner's inbox

```
try {
  await resend.emails.send({
    from: 'enquiries@aroraclasses.in',
    to: process.env.OWNER_EMAIL!,
    subject: `New enquiry – ${input.name}, Class ${input.student_class}`,
    text: `${input.phone}\n\n${input.message ?? ''}`,
  });
} catch (e) {
  logger.error('email failed', e); // row already saved
}
```

5 She sees a confirmation

With the centre's phone number as a fallback, so a parent in a hurry can still call rather than wait.

RESPONSE

200 OK

```
{ "ok": true,
  "data": { "id": "6f1c...",
            "created_at": "2026-04-12T09:14:22Z" } }
```

```
// UI: "Thanks – we'll call you back today.
//      In a hurry? Call 98xxx xxxxx."
```

The guarantee this design makes. The enquiry row is the source of truth, not the email. If Resend is down, the parent still gets a success message and the owner finds the enquiry waiting in the admin inbox. Storing and notifying are two different promises, and only one of them is allowed to fail.

The owner logs in

One account, one password. There is no signup, no password reset by email, and no second user — adding them would be scope with no beneficiary.

1 He enters the password

The password itself is never stored anywhere. Only a bcrypt hash exists, as an environment variable on the server.

PATH B · AUTH

POST /api/auth/login

```
// Request
{ "password": "*****" }

const ok = await bcrypt.compare(
  body.password,
  process.env.ADMIN_PASSWORD_HASH!
);
if (!ok) return json({ ok:false,
  error:{ code:'BAD_CREDENTIALS' } }, 401);
```

2 A signed cookie is issued

`httpOnly` means JavaScript cannot read it, so a script injected into the page still cannot steal the session.

SESSION

Set-Cookie: session=...

```
cookies().set('session', await sign({ role:'admin' })), {
  httpOnly: true,
  secure: true,
  sameSite: 'lax',
  maxAge: 60 * 60 * 24 * 7, // 7 days
  path: '/',
});
```

3 Middleware guards every admin page

And every admin API route checks the session again on its own. Middleware protects pages; routes are reachable independently and must protect themselves.

GUARD

middleware.ts → /admin/*

```
export function middleware(req: NextRequest) {
  if (!req.nextUrl.pathname.startsWith('/admin')) return;
  if (req.nextUrl.pathname === '/admin/login') return;

  const valid = verify(req.cookies.get('session')?.value);
  if (!valid) return NextResponse.redirect(
    new URL('/admin/login', req.url));
}

export const config = { matcher: '/admin/:path*' };
```

The owner posts an update with a PDF

Two requests, not one: the file is uploaded first and returns a URL, then the update is created with that URL. Splitting them keeps the update endpoint simple and lets the upload be reused.

1 He picks a file on his phone

The server checks size and type — the browser's `accept` attribute is trivially bypassed, so it proves nothing. The original filename is discarded and replaced with a UUID.

PATH B · ADMIN

POST /api/upload

```
const ALLOWED = ['image/jpeg', 'image/png', 'image/webp',
  'application/pdf'];
const MAX = 5 * 1024 * 1024;

if (!ALLOWED.includes(file.type) || file.size > MAX)
  return json({ ok:false,
    error:{ code:'FILE_REJECTED' } }, 413);

const key = `updates/${crypto.randomUUID()}.${ext}`;

// Response - 200
{ "ok": true, "data": { "url": "https://.../update-files/
  updates/9a4f2c17-....pdf" } }
```

2 The update row is created

Title, body, and the URL returned by step 1. The file itself is never stored in the database — only the link to it.

DATA ACCESS

POST /api/updates

```
// Request
{
  "title": "Class 12 pre-board timetable",
  "body": "Pre-boards begin 8 January. Timetable attached.",
  "file_url": "https://.../updates/9a4f2c17-....pdf"
}

// Response - 201
{ "ok": true, "data": { "id": "b73e..." } }
```

3 Parents see it within a minute

The public Updates page revalidates on a 60-second timer. No redeploy, no intervention from you.

PATH A

revalidate = 60

```
// Deleting is soft — the row stays, the flag flips.
// A hard delete would orphan the file in storage
// and lose the record of what was announced.
```

```
PATCH /api/updates/b73e...
{ "is_published": false }
```

The owner works the enquiry inbox

The screen that stops the same parent being called twice, or not at all.

1 He opens the inbox

Newest first, filtered by status. A table with one dropdown per row — no charts, no dashboard. He will use it on a phone between batches.

PATH B · ADMIN

GET /api/enquiries?status=new

```
{ "ok": true, "data": [
  { "id": "6f1c...", "name": "Rajwinder Kaur",
    "phone": "9876543210", "student_class": "11",
    "message": "Is the evening physics batch open?",
    "status": "new",
    "created_at": "2026-04-12T09:14:22Z" }
]}
```

2 He calls, then marks it contacted

Three states only: new, contacted, closed. The constraint lives in the database, so an invalid status cannot be written even by a bug.

STATUS

PATCH /api/enquiries/:id

```
// Request
{ "status": "contacted" }

// Enforced in the schema, not just the code:
status text not null default 'new'
check (status in ('new','contacted','closed'))
```

Frontend layer — routes and rendering

Ten routes. Six public and statically generated; four admin, dynamic and never cached. Only two pages in the whole application send data anywhere.

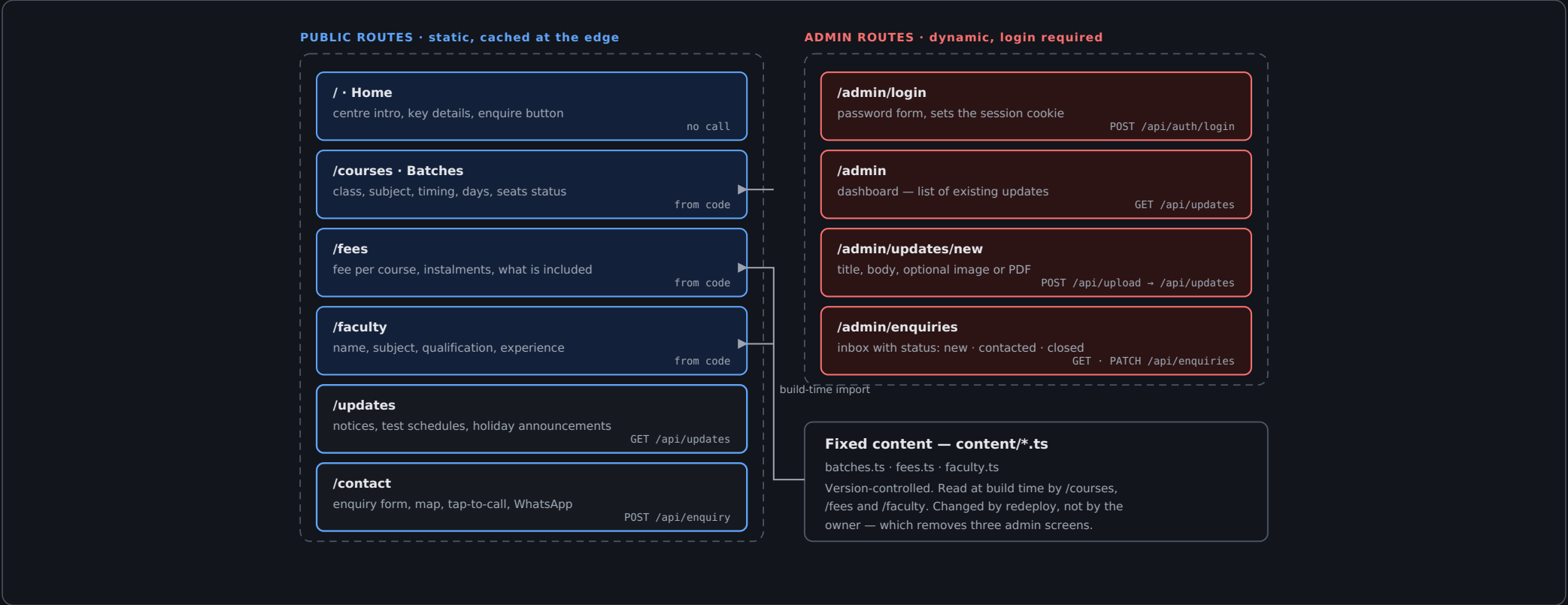


Figure 2 — Routes, rendering mode, and the call each route makes.

ROUTE	MODE	WHY
Home, Courses, Fees, Faculty	Static	Changes only on redeploy; served from CDN
Updates	Static + 60s revalidate	New posts must appear without a rebuild
Contact	Static shell, client form	Page is fixed; only the form is interactive
All <code>/admin/*</code>	Dynamic, uncached	Private data must never be cached

Backend layer — pipeline and endpoints

Four services, all serverless functions inside the same project. Every request passes the same five stages in the same order.

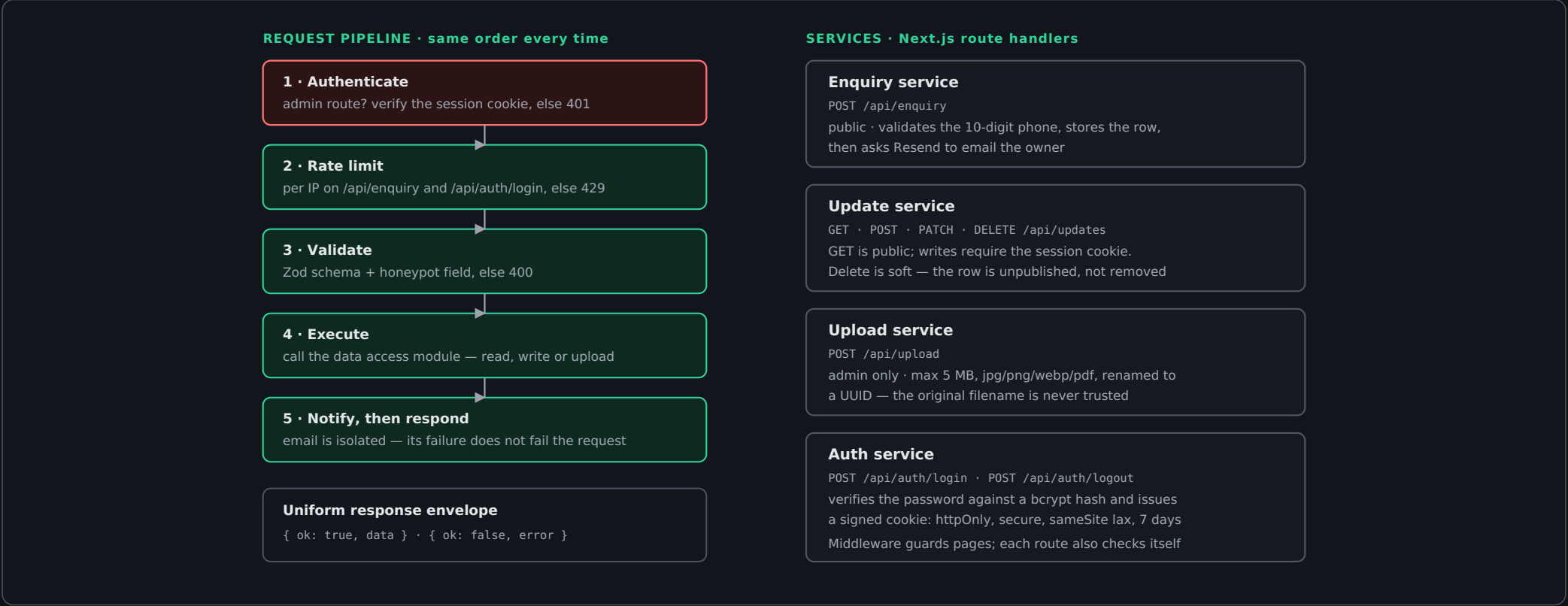


Figure 3 — Request pipeline and service responsibilities.

METHOD	ROUTE	AUTH	DOES
POST	/api/enquiry	public	Validate, persist, email the owner
GET	/api/updates	public	Published updates, newest first
POST	/api/updates	admin	Create an update
PATCH	/api/updates/:id	admin	Edit or unpublish
POST	/api/upload	admin	Store a file, return its URL
GET · PATCH	/api/enquiries	admin	List enquiries, change status
POST	/api/auth/login · /logout	public · admin	Issue or clear the session cookie

Status codes

CODE	WHEN	RETURNED BY
200 / 201	Success	All routes
400	Validation failed, or honeypot filled	Enquiry, Update
401	Missing or invalid session cookie	All admin routes
413	File over 5 MB or disallowed type	Upload
429	Rate limit exceeded for this IP	Enquiry, Login
500	Database unreachable	All routes

Database layer — tables, bucket, DDL

Two tables and one bucket. No relationship between the tables — they serve two unrelated flows. Batches, fees and faculty are not stored here at all.

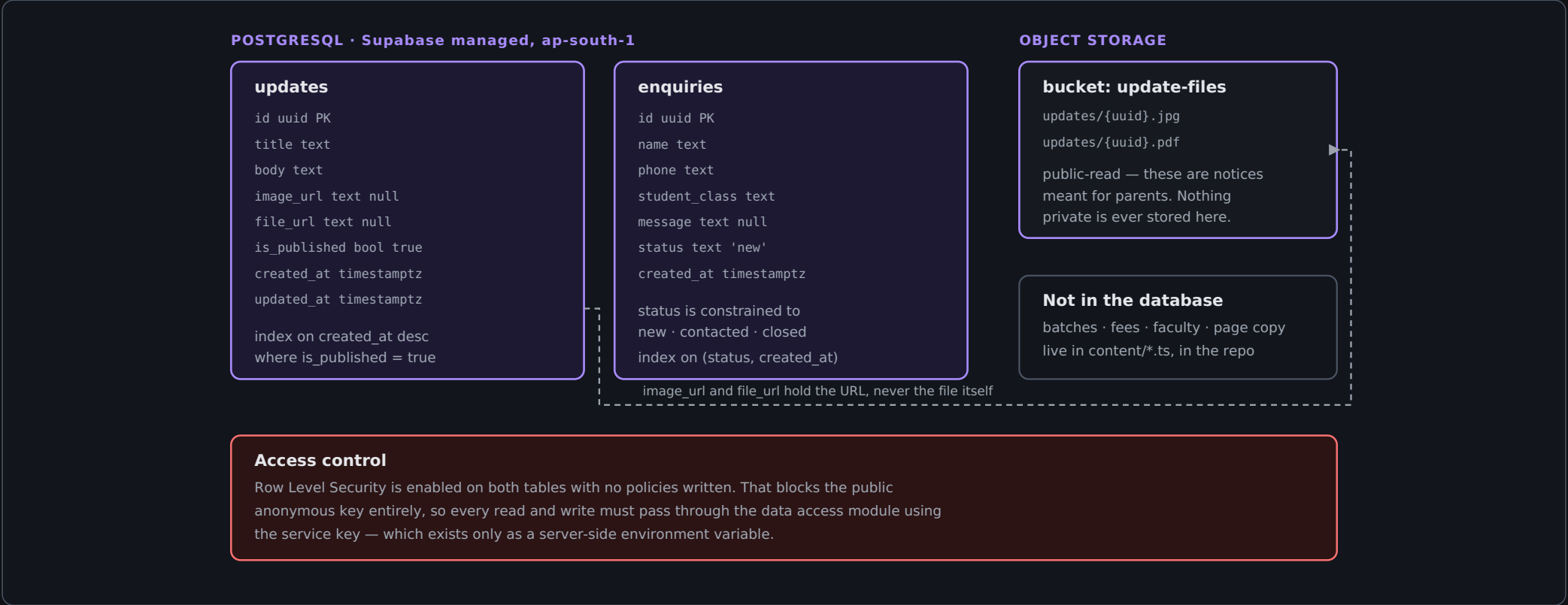


Figure 4 — Tables, bucket, and where the rest of the content lives.

TABLE	WRITTEN BY	READ BY	GROWTH
updates	Update service, admin only	Public Updates page	A few rows a month
enquiries	Enquiry service, public	Admin inbox only	10-20 rows a week in season
update-files	Upload service, admin only	Parent browsers, via CDN	Bounded by the free 1 GB

Schema — DDL

```
create table updates (  
  id          uuid primary key default gen_random_uuid(),  
  title       text not null,  
  body        text not null,  
  image_url   text,  
  file_url    text,  
  is_published boolean not null default true,  
  created_at  timestampz not null default now(),  
  updated_at  timestampz not null default now()  
);  
  
create index updates_public_idx on updates (created_at desc)  
  where is_published = true;  
  
create table enquiries (  
  id          uuid primary key default gen_random_uuid(),  
  name        text not null,  
  phone       text not null,  
  student_class text not null,  
  message     text,  
  status      text not null default 'new'  
              check (status in ('new','contacted','closed')),  
  created_at  timestampz not null default now()  
);  
  
create index enquiries_status_idx on enquiries (status, created_at desc);  
  
-- block the public anonymous key entirely; all access goes through  
-- the data access module using the service key  
alter table updates  enable row level security;  
alter table enquiries enable row level security;
```

Operational note. The Supabase free tier pauses a project after a week of inactivity. Open the dashboard once before any demo to wake it, and keep a CSV export of `enquiries` as a backup.

Security — three layers of defence

No single point of failure. If one layer is bypassed, the next one still holds.

LAYER	WHAT IT STOPS	WHERE IT LIVES
1 · Middleware	An unauthenticated browser reaching any admin page	<code>middleware.ts</code> , at the edge
2 · Route guard	A direct call to an admin API, bypassing the pages entirely	First lines of every admin handler
3 · Row Level Security	Anyone reaching the database with the public anonymous key	PostgreSQL itself

The boundary that matters most

Everything in the browser is public forever. Anyone can open the developer tools and read the JavaScript bundle. That single fact decides where every secret lives.

VARIABLE	REACHES THE BROWSER?	IF LEAKED
<code>NEXT_PUBLIC_SITE_URL</code>	Yes, by design	Harmless — it's the public address
<code>SUPABASE_SERVICE_KEY</code>	Never	Full read and write on every table
<code>RESEND_API_KEY</code>	Never	Anyone can send mail as the centre
<code>SESSION_SECRET</code>	Never	Session cookies can be forged
<code>ADMIN_PASSWORD_HASH</code>	Never	Offline attack on the password

If a secret ever lands in a commit, rotate it. Deleting the line later does not help — git keeps the history, and a public repository is scraped within minutes. Write `.gitignore` before the first commit, not after.

Other measures

- **Honeypot field** on the enquiry form — hidden from people, filled by bots, rejected server-side.
- **Per-IP rate limit** on the enquiry and login routes, so the inbox stays usable and the password can't be brute-forced.
- **Upload constraints** enforced on the server: 5 MB, four MIME types, filename replaced with a UUID.
- **Output sanitising** on update text, so a pasted fragment of HTML can't break or hijack the page.

Validation — one schema, both sides

The same Zod schema is imported by the form and by the API route. There is no chance of the browser and the server disagreeing about what a valid phone number is, because there is only one definition of it.

```
// lib/schemas.ts – imported by both the form and the route handler
export const EnquirySchema = z.object({
  name:      z.string().min(2).max(60),
  phone:     z.string().regex(/^?[6-9]\d{9}$/), // Indian mobile
  student_class: z.enum(['9','10','11','12','other']),
  message:    z.string().max(500).optional(),
  website:    z.string().max(0),               // honeypot
});

export const UpdateSchema = z.object({
  title:      z.string().min(3).max(120),
  body:       z.string().min(1).max(4000),
  image_url:  z.string().url().optional(),
  file_url:   z.string().url().optional(),
});
```

RULE	ENFORCED IN THE BROWSER	ENFORCED ON THE SERVER	ENFORCED IN THE DATABASE
Phone is 10 digits, starts 6–9	Yes, for instant feedback	Yes — the one that counts	—
Class is one of five values	Yes, it's a dropdown	Yes	—
Status is new/contacted/closed	Yes, it's a dropdown	Yes	Yes — ☐ check constraint
File ≤ 5 MB, four MIME types	Hint only	Yes — the one that counts	—

Why repeat it. Browser validation exists so a parent sees her mistake before tapping Submit. It is not security: anyone can call the endpoint directly with curl. The server check is the real one, and the database constraint is the last line.

Deployment — where each part runs

One repository, one deploy. Pushing to a branch produces a preview URL; merging to `main` promotes it to production. Rollback is one click.

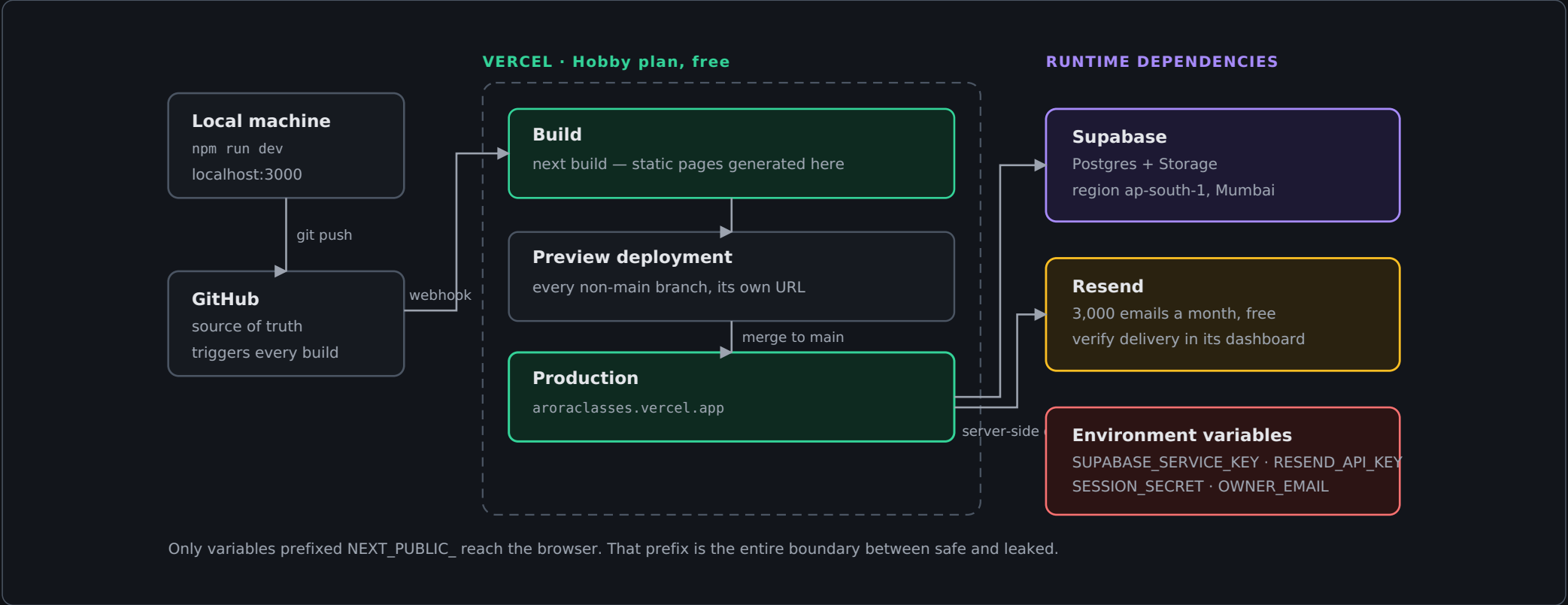


Figure 5 — Where each part physically runs, and where secrets live.

CONCERN	PROVIDER	PLAN	REGION
Hosting, CDN, serverless functions	Vercel	Hobby, free	ap-south-1
Database and object storage	Supabase	Free — 500 MB + 1 GB	ap-south-1, Mumbai
Transactional email	Resend	Free — 3,000/month	—
Source control and CI trigger	GitHub	Free	—

Why Mumbai. Co-locating the functions and the database in ap-south-1 removes a round trip across continents. Parents in Ludhiana on mobile data feel that difference.

Key decisions and roadmap

Each row below is a choice that could reasonably have gone the other way.

DECISION	RATIONALE	TRADE-OFF ACCEPTED
Monolith, not separate frontend and backend	One deploy, no CORS, no second host	They scale together
Serverless, not a persistent server	Zero cost at this traffic, no administration	Cold start on the first request
Batches and fees in code, not the database	Owner only edits updates; removes three tables and three admin screens	A fee change needs a redeploy
Enquiries stored <i>and</i> emailed	Email is unreliable; the row is the source of truth	Two places to check
Files in object storage, not Postgres	Databases handle binaries poorly; CDN serves them better	Two systems to keep consistent
RLS enabled with no policies	Stricter and simpler than row rules for a single admin	All access must go through the server
Email, not the WhatsApp API	Works on day one without business-number approval	Owner checks email less often
Soft delete on updates	Keeps the record of what was announced; avoids orphaned files	Rows accumulate, harmlessly

Roadmap — five phases, each independently deployable

PHASE	SCOPE	VALUE DELIVERED
1	Static site with all fixed content	Parents find timings and fees without calling
2	Enquiry form: validation, persistence, email	Missed calls become captured enquiries
3	Login and text-only updates	Owner publishes notices himself
4	Image and PDF attachments	Timetables and notices shared as files
5	Admin enquiry inbox with status	No enquiry followed up twice or forgotten

Ship phase 1 completely before starting phase 2. A finished phase 1 is a working product the owner can use today. Three half-built phases are not.